

Ex. No: 03

Histogram Equalization Using OpenCV (Grayscale & Color Images)

Name : ASWINI D **Reg. No :** 212225240015 **Slot No :** OD1143

Write a Python program using OpenCV to perform histogram equalization on the given grayscale image "parrot.jpg".

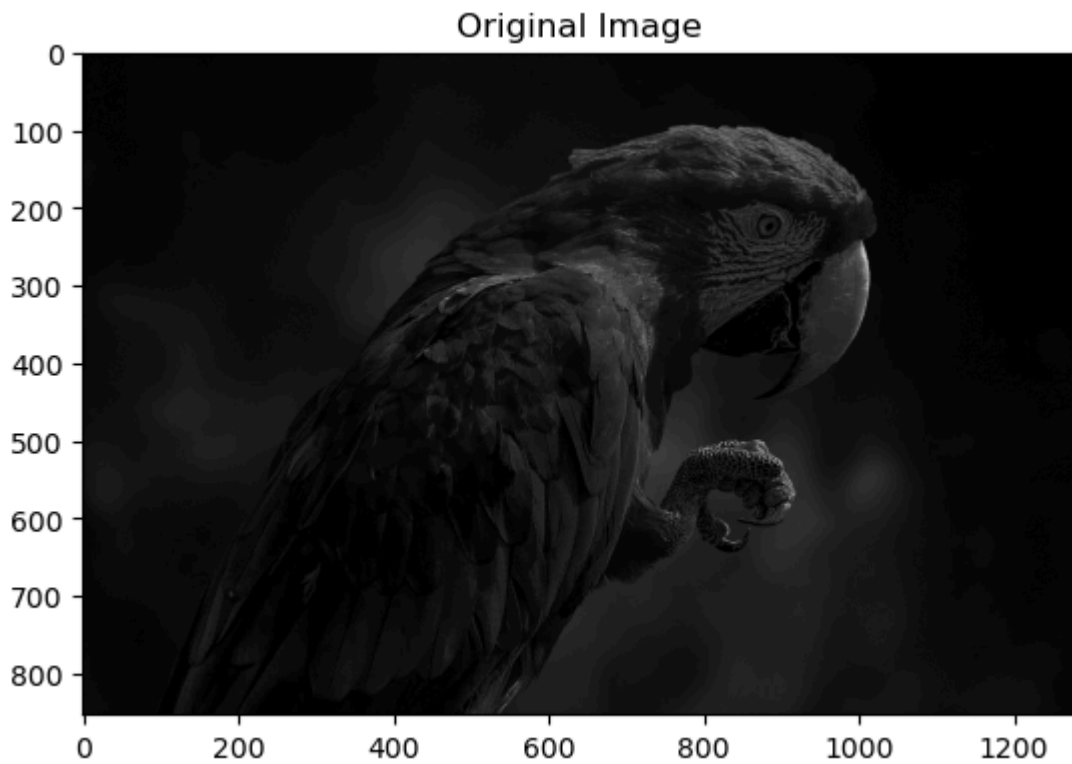
The program should:

- Import the required libraries (**OpenCV, NumPy, Matplotlib**).
- Read the image "**parrot.jpg**" in **grayscale** format.
- Display the grayscale image and plot its histogram.
- Perform **histogram equalization** on the grayscale image using `cv2.equalizeHist()` to enhance its contrast.
- Using Matplotlib, display the following in a **2 × 2 grid layout**:
 - Original Grayscale Image
 - Histogram of Original Image
 - Enhanced (Equalized) Image
 - Histogram of Enhanced Image

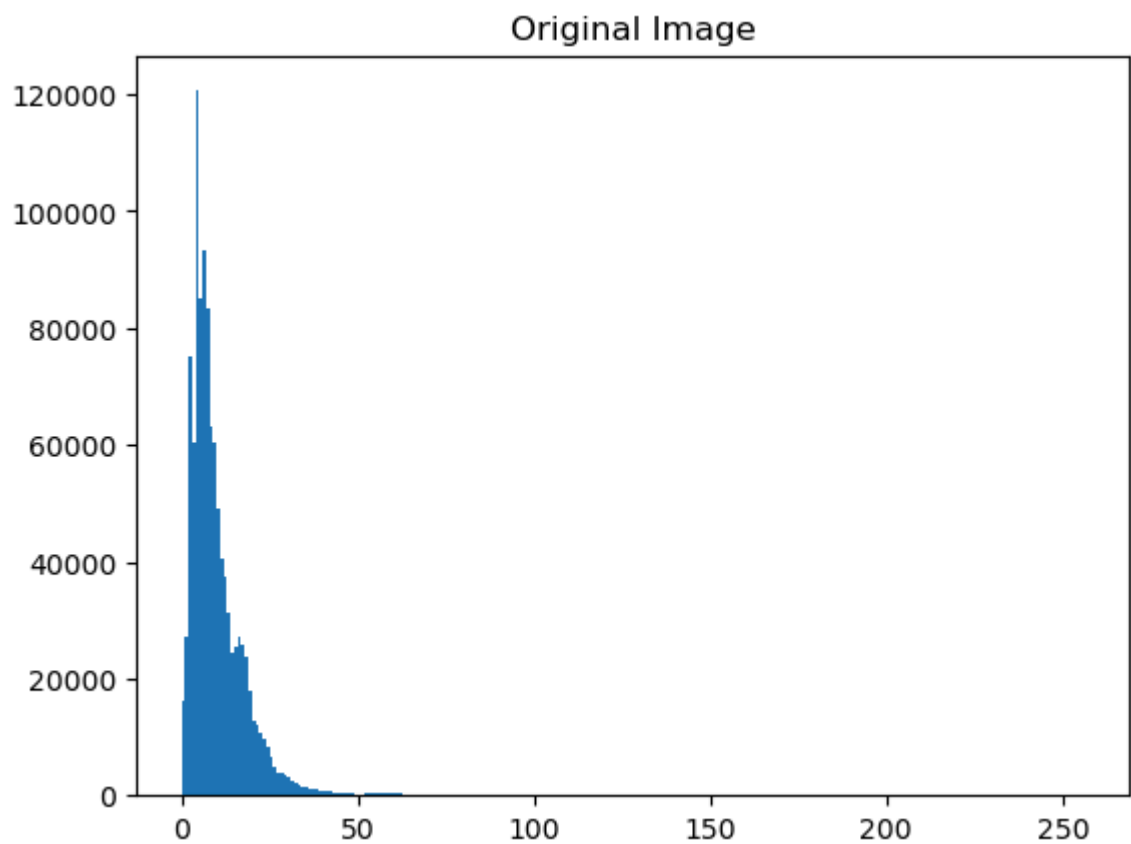
```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
In [4]: # Read the image in grayscale format.
img = cv2.imread('parrot.jpg', cv2.IMREAD_GRAYSCALE)
```

```
In [5]: # Display the images.
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.show()
```

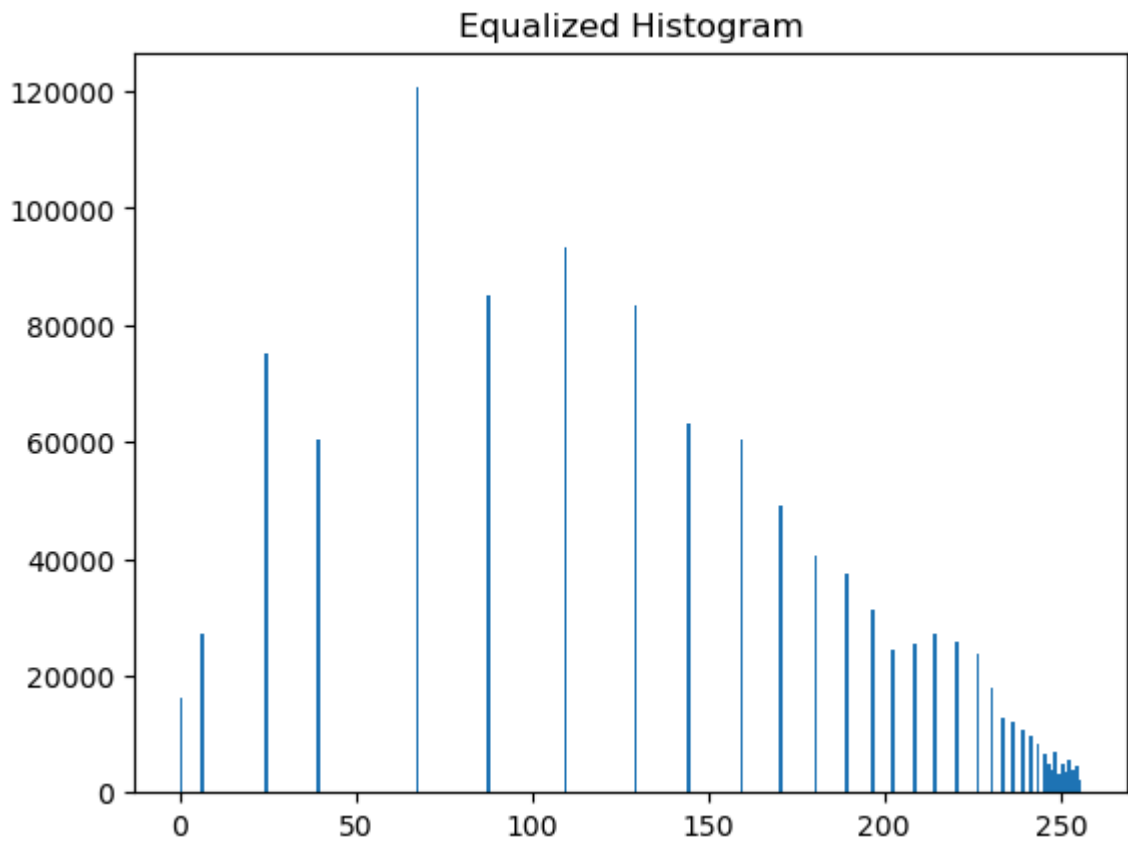


```
In [7]: # Display the images
plt.hist(img.ravel(),256,range = [0, 256]);
plt.title('Original Image')
plt.show()
```

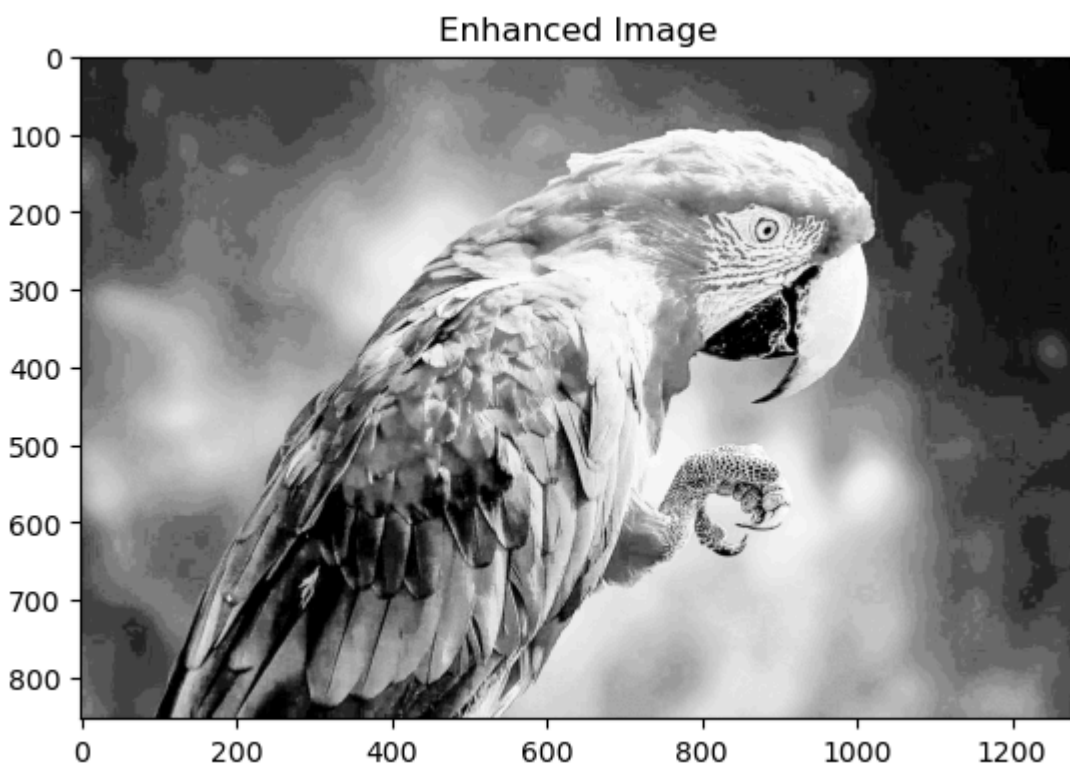


```
In [8]: # Perform histogram equalization
img_eq = cv2.equalizeHist(img)
plt.hist(img_eq.ravel(), 256, range = [0, 256])
plt.title('Equalized Histogram')
```

```
Out[8]: Text(0.5, 1.0, 'Equalized Histogram')
```



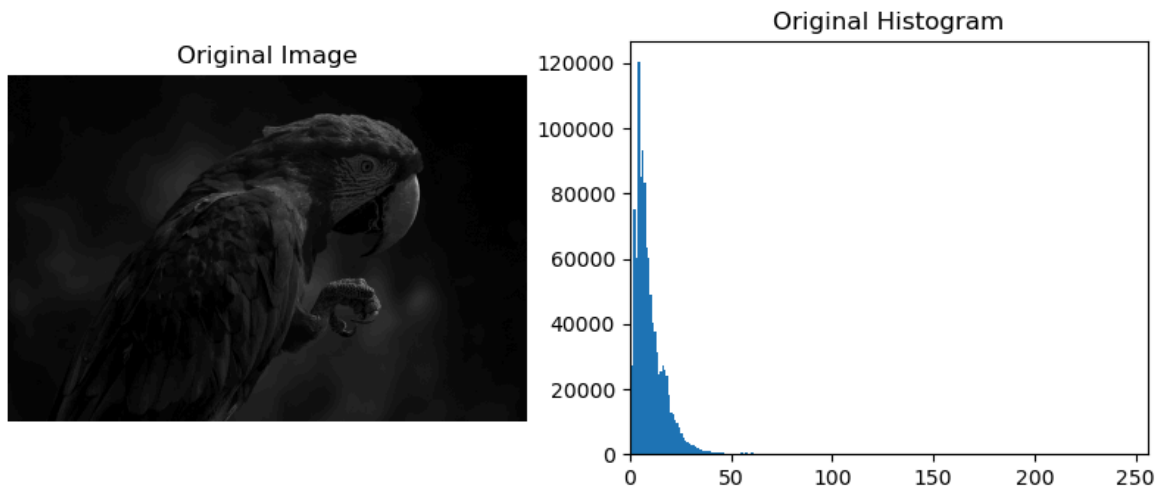
```
In [9]: plt.imshow(img_eq, cmap='gray')  
plt.title('Enhanced Image')  
plt.show()
```



```
In [10]: # Display [1] the Original Image (Gray Image) and its Histogram.  
plt.figure(figsize=(10, 8))  
# Original Image  
plt.subplot(2, 2, 1)
```

```
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")
# Original Histogram
plt.subplot(2, 2, 2)
plt.hist(img.ravel(), bins=256, range=[0, 256])
plt.title("Original Histogram")
plt.xlim([0, 256])
```

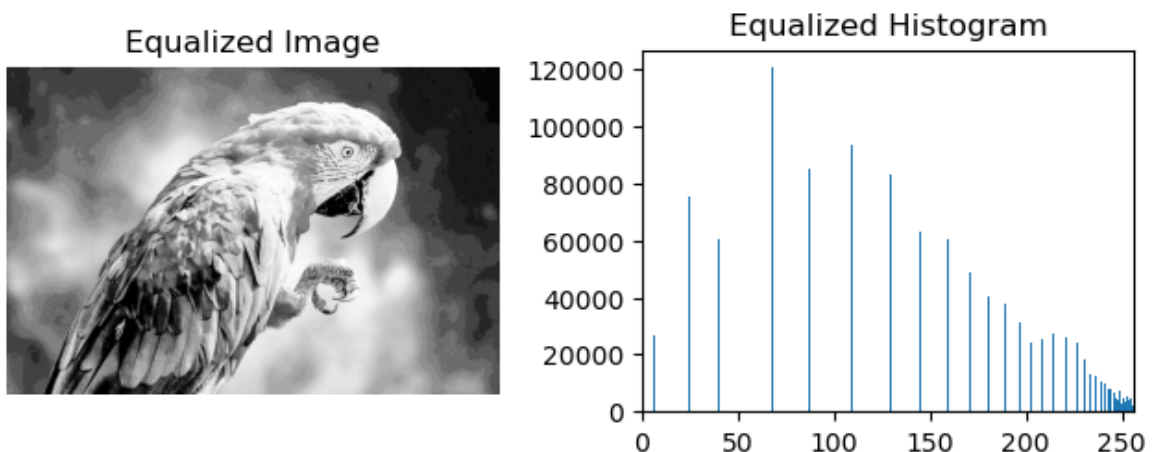
Out[10]: (0.0, 256.0)



In [11]: # [2] the Enhanced Image and its Histogram using a 2x2 Layout in Matplotlib.

```
# Equalized Image
plt.subplot(2, 2, 3)
plt.imshow(img_eq, cmap='gray')
plt.title("Equalized Image")
plt.axis("off")
# Equalized Histogram
plt.subplot(2, 2, 4)
plt.hist(img_eq.ravel(), bins=256, range=[0, 256])
plt.title("Equalized Histogram")
plt.xlim([0, 256])

plt.tight_layout()
plt.show()
```



Write a Python program using OpenCV to perform color image enhancement through histogram equalization in the

HSV color space on the given image "parrot.jpg".

The program should:

- Import the required libraries (**OpenCV, NumPy, Matplotlib**).
- Read the color image "**parrot.jpg**".
- Plot the **histogram of the original color image** (use the **B, G, and R** channels).
- Convert the image from **BGR to HSV** color space.
- Apply **histogram equalization** to one specific channel to improve brightness and contrast using `cv2.equalizeHist()` .
- Convert the enhanced HSV image back to **BGR** format.
- Using Matplotlib, display the following in a **2 × 2 grid layout**:
 - Original Color Image
 - Histogram of Original Image (BGR channels)
 - Enhanced Color Image
 - Histogram of Enhanced Image (BGR channels)

```
In [12]: # Import required libraries
import cv2
import matplotlib.pyplot as plt
```

```
In [13]: # Read the color given parrot.jpg image.
img1 = cv2.imread('parrot.jpg', cv2.IMREAD_COLOR)
plt.imshow(img1)
plt.title('Colour Image')
plt.axis("off")
plt.show()
```

Colour Image

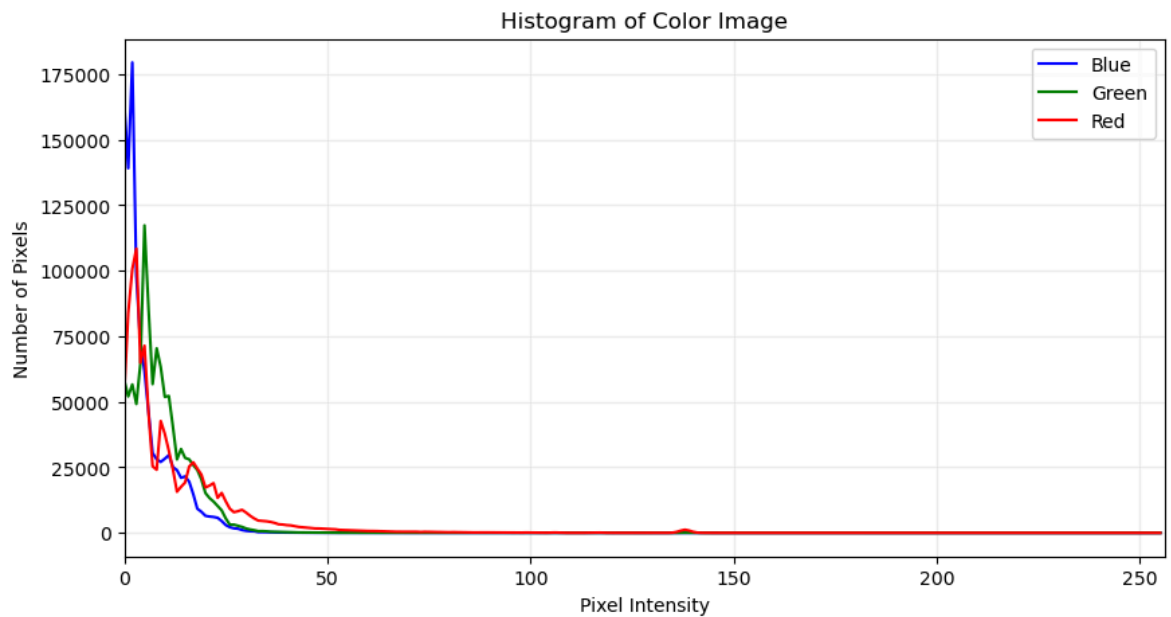


```
In [15]: plt.figure(figsize=(10, 5))

hist_b = cv2.calcHist([img1], [0], None, [256], [0, 256])
hist_g = cv2.calcHist([img1], [1], None, [256], [0, 256])
hist_r = cv2.calcHist([img1], [2], None, [256], [0, 256])
```

```
plt.plot(hist_b, color='b', label='Blue')
plt.plot(hist_g, color='g', label='Green')
plt.plot(hist_r, color='r', label='Red')

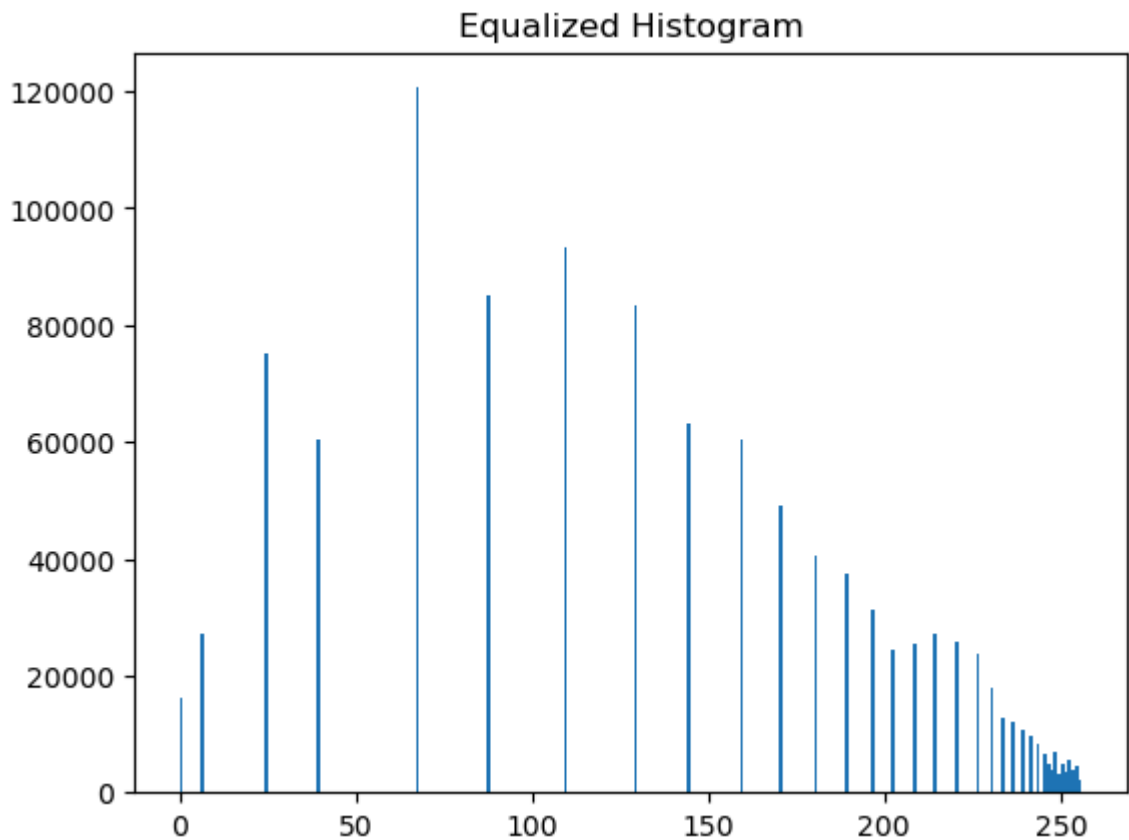
plt.title("Histogram of Color Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")
plt.xlim([0, 256])
plt.grid(alpha=0.2)
plt.legend()
plt.show()
```



```
In [17]: # Convert to HSV.
img_hsv = cv2.cvtColor(img1, cv2.COLOR_BGR2HSV)
```

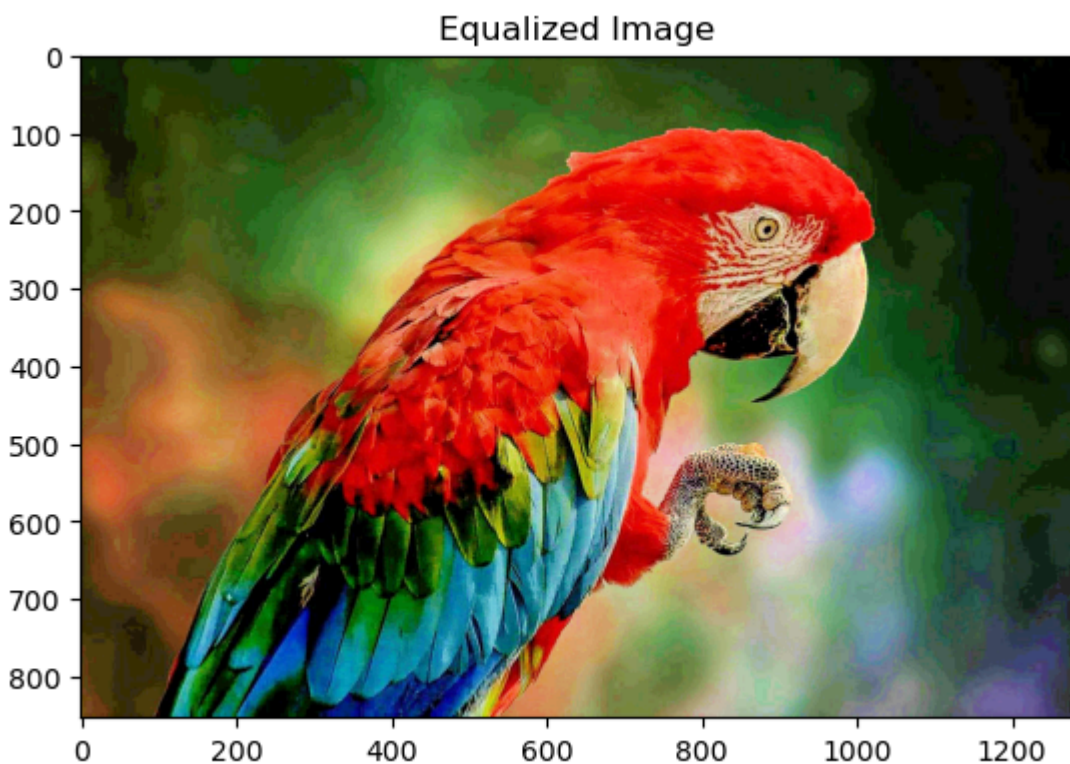
```
In [18]: # Perform histogram equalization
img_hsv[:, :, 2] = cv2.equalizeHist(img_hsv[:, :, 2])
plt.hist(img_eq.ravel(), 256, range = [0, 256])
plt.title('Equalized Histogram')
```

```
Out[18]: Text(0.5, 1.0, 'Equalized Histogram')
```



```
In [19]: # Convert back to BGR format  
img_eq = cv2.cvtColor(img_hsv, cv2.COLOR_HSV2BGR)
```

```
In [20]: # Display  
plt.imshow(img_eq[:, :, ::-1]); plt.title('Equalized Image'); plt.show()
```



```
In [23]: # Convert BGR image to RGB for Matplotlib display  
image_rgb = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
```

```

# Display image and its histogram
plt.figure(figsize=(10, 8))

# Original Image
plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Colour Image")
plt.axis("off")

# Original Histogram
plt.subplot(2, 2, 2)

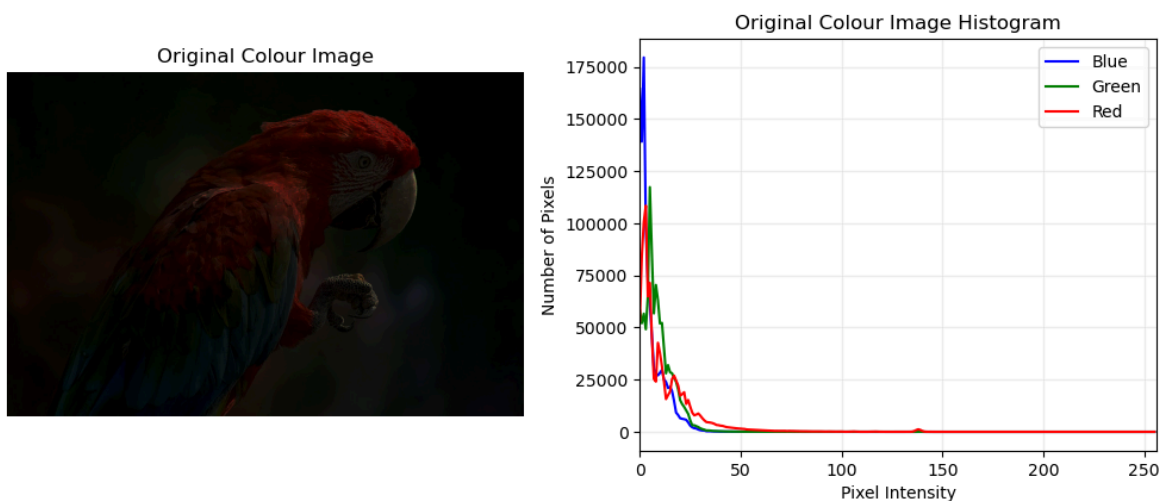
hist_b = cv2.calcHist([img1], [0], None, [256], [0, 256])
hist_g = cv2.calcHist([img1], [1], None, [256], [0, 256])
hist_r = cv2.calcHist([img1], [2], None, [256], [0, 256])

plt.plot(hist_b, color='b', label='Blue')
plt.plot(hist_g, color='g', label='Green')
plt.plot(hist_r, color='r', label='Red')

plt.title("Original Colour Image Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")
plt.xlim([0, 256])
plt.legend()
plt.grid(alpha=0.2)

plt.tight_layout()
plt.show()

```



```

In [35]: # Equalized Image
plt.subplot(223)
plt.imshow(img_eq[:, :, ::-1])
plt.title("Equalized Image")
plt.axis("off")

# Equalized Histogram
plt.subplot(224)
plt.hist(img_eq.ravel(), bins=256, range=[0, 256])
plt.title("Equalized Histogram")
plt.xlim([0, 256])

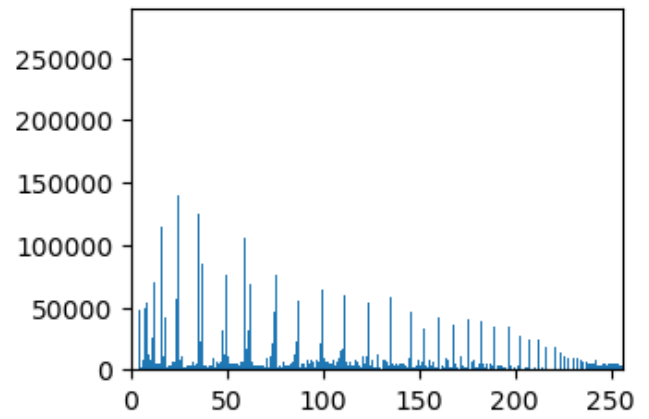
plt.tight_layout()
plt.show()

```


Equalized Image



Equalized Histogram



In []: